

Mathematical Explanation of the Spectral Solver for Shu (1977) Self-Similar Collapse

Numerical cross-validation project

October 8, 2026

Abstract

This document explains the mathematical content of each source file of an independent Chebyshev spectral solver for the self-similar collapse equations of Shu (1977, *ApJ* **214**, 488). The code is written in GNU Octave and reproduces Shu’s Tables 1 and 2. The purpose of this document is to make the numerical implementation traceable from the published equations to the corresponding source-code operations. The Runge–Kutta integration used by Shu is replaced here by a Chebyshev collocation formulation followed by a nonlinear Newton solve. The resulting solution reproduces the tabulated values of Shu (1977) with a maximum relative difference of 0.293% for Table 1 and 0.618% for Table 2.

Contents

1	Introduction (Governing equations)	3
2	Numerical formulation	3
3	Logarithmic change of variables	4
3.1	Logarithmic coordinate	4
3.2	Log-density variable	4
4	The Chebyshev differentiation matrix — <code>sc_cheb.m</code>	5
4.1	Mapping between ξ and τ	5
5	The residual equations	6
5.1	Collapse problem — <code>sc_res_collapse.m</code>	6
5.1.1	Outer asymptotic boundary condition	6
5.2	Expansion-wave problem — <code>sc_res_expwave.m</code>	7
6	The Newton solver — <code>sc_newton_cs.m</code>	7
6.1	Complex-step Jacobian	7
6.2	Row scaling	8
6.3	Tikhonov-regularized least-squares Newton step	8
6.4	Backtracking line search	8
6.5	Convergence criterion	9
7	Grid refinement and continuation	9
7.1	Grid refinement in N	9
7.2	Continuation in A	9
7.3	Initial guess	10

8	Method and code provenance	10
9	Equation-to-code traceability	11
10	The main driver — <code>shu1977_spectral_octave.m</code>	13
11	Numerical validation	13
12	Summary of the method	14
13	Source-file summary	15
14	Reproducibility statement	15

1 Introduction (Governing equations)

Shu (1977) [1] writes the similarity ansatz

$$\rho(r, t) = \frac{\alpha(x)}{4\pi G t^2}, \quad u(r, t) = a v(x), \quad M(r, t) = \frac{a^3 t}{G} m(x), \quad x = \frac{r}{at}, \quad (1)$$

where a is the isothermal sound speed, G the gravitational constant, ρ the mass density, u the radial velocity (negative for inflow), M the mass interior to radius r , and t the physical time. Substituting into the continuity and Euler equations yields the two coupled ordinary differential equations (Shu 1977, eqs. 11–12):

$$\boxed{[(x - v)^2 - 1] \frac{dv}{dx} = \left[\alpha(x - v) - \frac{2}{x} \right] (x - v)} \quad (2)$$

$$\boxed{[(x - v)^2 - 1] \frac{1}{\alpha} \frac{d\alpha}{dx} = \left[\alpha - \frac{2(x - v)}{x} \right] (x - v)} \quad (3)$$

together with the algebraic mass relation (Shu 1977, eq. 10)

$$m(x) = x^2 \alpha(x) (x - v(x)). \quad (4)$$

The coefficient multiplying the derivatives vanishes on the *critical (sonic) curve*

$$(x - v)^2 = 1 \quad \Longleftrightarrow \quad v = x - 1 \quad (\text{for } x - v = 1). \quad (5)$$

A physical solution must cross this curve smoothly. This requires the right-hand sides of Eq. (2)–(3) to vanish at the crossing point as well, so the singularity of the equations is removable for the physical solution.

The present paper is organized as follows. Section 2 summarizes the overall numerical strategy. Section 3 introduces the logarithmic coordinate $\tau = \ln x$ and the log-density variable $w = \ln \alpha$ and derives the reformulated similarity equations. Section 4 constructs the Chebyshev differentiation matrix `sc_cheb.m`. Section 5 assembles the collocation residuals for the collapse and expansion-wave problems. Sections 6–7 describe the Newton solver, the complex-step Jacobian, the Tikhonov-regularized Newton step, the Armijo backtracking, and the continuation strategies in N and A . Section 9 provides an equation-to-code traceability table, and Sections 10–11 present the main driver, the numerical validation, and the interpretation of the observed differences from Shu (1977). Sections 12–14 conclude with a summary of the method, a source-file listing, and a reproducibility statement.

2 Numerical formulation

The numerical procedure of Shu (1977) starts from asymptotic conditions and integrates the ordinary differential equations with a four-step Runge–Kutta method. The present implementation instead follows

$$\boxed{\text{Shu equations} \longrightarrow \text{spectral collocation} \longrightarrow \text{nonlinear residual equations} \longrightarrow \text{Newton solve.}} \quad (6)$$

It does not reproduce the Runge–Kutta algorithm line by line. It solves the same mathematical equations with a different discretization (Chebyshev collocation [2, 3]) and a different nonlinear solution method (Newton’s method [4]). This difference is what makes the comparison a meaningful cross-check: the two sets of numerical solutions were obtained independently. Two changes of variables (Sec. 3) make the nonlinear boundary-value problem more suitable for spectral collocation [3].

The implementation therefore combines two distinct kinds of provenance: (1) the physical equations, asymptotic conditions, and tabulated reference values are taken from Shu (1977), while (2) the numerical algorithms used to solve those equations are standard methods documented in the cited numerical-analysis literature. The source file `sc_cheb.m` contains an implementation based on Trefethen’s Program 6 [2]; the remaining solver routines are independent implementations of the cited numerical methods rather than recovered versions of Shu’s original software.

3 Logarithmic change of variables

3.1 Logarithmic coordinate

The similarity variable x spans several decades ($x_{\min} \sim 10^{-3}$ and $x_{\max} \sim 20$ in the collapse calculation). Because t denotes physical time in Eq. (1), the logarithmic coordinate is called τ here (it is called `t` in the code, hence `t_min`, `t_max` and `Dt`):

$$\tau = \ln x, \quad \frac{d}{dx} = \frac{1}{x} \frac{d}{d\tau}. \quad (7)$$

The computational domain is the finite interval

$$[\tau_{\min}, \tau_{\max}] = [\ln x_{\min}, \ln x_{\max}],$$

which is mapped linearly onto the Chebyshev interval $[-1, 1]$. A grid in τ spaces its nodes by equal ratios in x , so each decade of x is resolved comparably; a grid uniform in x would place almost all nodes at large x . (The Chebyshev nodes in τ additionally cluster near both ends of the interval.) The use of logarithmic coordinates to compress multi-decade ranges is standard in pseudospectral methods [3].

3.2 Log-density variable

The reduced density has the asymptotic behavior derived from the self-similar solutions discussed by Shu (1977) [1]:

$$\alpha(x) \sim \sqrt{\frac{m_0}{2x^3}} \quad (x \rightarrow 0), \quad (8)$$

and

$$\alpha(x) \sim \frac{A}{x^2} \quad (x \rightarrow \infty). \quad (9)$$

Here $m_0 = \lim_{x \rightarrow 0} m(x)$ is the dimensionless mass-accretion parameter, so that $dM/dt = m_0 a^3/G$ at small radii, and A is the coefficient of the large- x asymptote. Since α becomes large toward the small- x end of the domain, the code uses

$$w(x) = \ln \alpha(x), \quad \alpha = e^w. \quad (10)$$

This has two numerical advantages:

1. positivity of α is automatic because $e^w > 0$;
2. the dynamic range of the unknown density is reduced.

Using Eqs. (7) and (10), the original ODE system becomes

$$\boxed{[(x-v)^2 - 1] \frac{dv}{d\tau} = [\alpha x(x-v) - 2](x-v)} \quad (11)$$

and

$$\boxed{[(x-v)^2 - 1] \frac{dw}{d\tau} = [\alpha x - 2(x-v)](x-v),} \quad (12)$$

where

$$x = e^\tau, \quad \alpha = e^w. \quad (13)$$

These are the equations enforced by the spectral residual functions.

4 The Chebyshev differentiation matrix — `sc_cheb.m`

The file `sc_cheb.m` constructs the Chebyshev differentiation matrix. The implementation follows the standard barycentric-weight construction of Trefethen [2, Program 6]; the corresponding source code is included in this project for self-contained reproducibility. This is explicitly identified as third-party algorithmic/code provenance, not as an independently invented differentiation-matrix construction. (The variable `x` in this listing holds the nodes ξ_j , not the similarity variable.)

```

1 function [D, x] = sc_cheb(N)
2     x = cos(pi * (0:N)' / N);
3     c = [2; ones(N-1,1); 2] .* (-1).^(0:N)';
4     X = repmat(x, 1, N+1);
5     dX = X - X';
6     D = (c * (1./c)') ./ (dX + eye(N+1));
7     D = D - diag(sum(D, 2));
8 end

```

The nodes are the Chebyshev–Gauss–Lobatto nodes

$$\xi_j = \cos\left(\frac{\pi j}{N}\right), \quad j = 0, \dots, N. \quad (14)$$

They cluster near $\xi = \pm 1$, which is useful when high accuracy is required near the endpoints of the computational interval [2, 3]. For a vector of nodal values $\mathbf{f}$, the differentiation matrix satisfies

$$(D\mathbf{f})_i \approx \left. \frac{df}{d\xi} \right|_{\xi_i}. \quad (15)$$

The implementation uses the standard barycentric-weight construction [2, Ch. 6]. The final diagonal correction makes the derivative of a constant function numerically zero.

4.1 Mapping between ξ and τ

The computational coordinate τ is related linearly to $\xi \in [-1, 1]$:

$$\tau = \frac{\tau_{\max} - \tau_{\min}}{2} \xi + \frac{\tau_{\max} + \tau_{\min}}{2}. \quad (16)$$

Therefore,

$$\frac{d}{d\tau} = \frac{2}{\tau_{\max} - \tau_{\min}} \frac{d}{d\xi}. \quad (17)$$

The code constructs

```

1 Dt = D * 2 / (t_max - t_min);

```

so that

$$D_\tau \mathbf{f} \approx \frac{d\mathbf{f}}{d\tau}. \quad (18)$$

The nodes are ordered from $\xi_0 = +1$ (that is, $\tau_{\max}$ and $x_{\max}$) to $\xi_N = -1$ ($\tau_{\min}$ and $x_{\min}$). Node $j = 0$ is therefore the outer boundary.

5 The residual equations

The spectral method [2, 3] replaces the differential equations with a finite-dimensional nonlinear algebraic system. Bold symbols denote vectors of nodal values; plain v , w denote the functions. The unknown continuous profiles are discretized by their nodal values

$$\mathbf{v} = (v_0, \dots, v_N)^T, \quad \mathbf{w} = (w_0, \dots, w_N)^T, \quad (19)$$

where $w_j = \ln \alpha_j$. These are concatenated into a single state vector of $2(N+1)$ nodal unknowns

$$\mathbf{u} = \begin{bmatrix} \mathbf{v} \\ \mathbf{w} \end{bmatrix}. \quad (20)$$

These are nodal values, not expansion coefficients of a Chebyshev polynomial. The differentiation matrix provides the derivatives of the interpolating polynomial at the collocation nodes.

Two features of the formulation are essential. First, the residuals are enforced at every node except the outer one, including nodes near the critical curve (5) where the coefficient of the derivative vanishes; this forces the interpolating polynomial to satisfy the regularity condition there, which selects the physical solution. Second, no boundary condition is imposed at $x_{\min}$. The behavior at small x is a result of the calculation, and it is compared with the free-fall behavior

$$\alpha \propto x^{-3/2}, \quad v \propto -x^{-1/2} \quad (x \rightarrow 0). \quad (21)$$

Indices: the nodes are numbered $j = 0, \dots, N$ in the text, while the code uses $1, \dots, N+1$, so the outer node $j = 0$ is element (1) in the code.

5.1 Collapse problem — `sc_res_collapse.m`

The residual function first computes the spectral derivatives. Here $\mathbf{x}$ is the vector of nodal similarity coordinates $x = e^\tau$, $\mathbf{alpha}$ is the vector e^w , and $\mathbf{Dt}$ is the matrix of Eq. (18):

```

1 dv = Dt * v;
2 dw = Dt * w;
3 xv = x - v;
4 coef = xv.^2 - 1;
5
6 R1 = coef .* dv - (alpha .* x .* xv - 2) .* xv;
7 R2 = coef .* dw - (alpha .* x - 2 .* xv) .* xv;
```

These expressions correspond directly to

$$R_1 = [(x - v)^2 - 1] \frac{dv}{d\tau} - [\alpha x(x - v) - 2](x - v), \quad (22)$$

$$R_2 = [(x - v)^2 - 1] \frac{dw}{d\tau} - [\alpha x - 2(x - v)](x - v). \quad (23)$$

At every node other than the outer one, the numerical solution must satisfy

$$R_1 = 0, \quad R_2 = 0. \quad (24)$$

5.1.1 Outer asymptotic boundary condition

At the outer node $j = 0$ ($x = x_{\max}$), the code replaces the two ODE residuals by the asymptotic boundary conditions, Eq. (19) of Shu (1977) [1]:

```

1 alpha_inf = A/x_max^2 - A*(A-2)/(2*x_max^4);
2 v_inf     = -(A-2)/x_max - (1-A/6)*(A-2)/x_max^3;
3
4 R1(1) = v(1) - v_inf;
5 R2(1) = w(1) - log(alpha_inf);
```

Accordingly,

$$R_1|_{j=0} = v_0 - \left[-\frac{A-2}{x_{\max}} - \frac{(1-A/6)(A-2)}{x_{\max}^3} \right], \quad (25)$$

$$R_2|_{j=0} = w_0 - \ln \left[\frac{A}{x_{\max}^2} - \frac{A(A-2)}{2x_{\max}^4} \right]. \quad (26)$$

These equations place the asymptotic solution at the truncated outer boundary.

5.2 Expansion-wave problem — `sc_res_expwave.m`

For $A = 2$ the outer asymptote (9) reduces to the static singular isothermal sphere $\alpha = 2/x^2$, $v = 0$. The expansion-wave solution is the limiting case $A \rightarrow 2^+$ of the collapse family [1]. Its outer boundary is the critical point $x = 1$, and the solution is computed on $x_{\min} \leq x \leq 1$ with $x_{\min} = 10^{-3}$. The boundary conditions at the outer node are

```
1 R1(1) = v(1);
2 R2(1) = w(1) - log(2);
```

that is,

$$v(x=1) = 0, \quad \alpha(x=1) = 2. \quad (27)$$

These are the critical-point values of the expansion-wave solution. With them, Eq. (4) gives $m(x=1) = 2$. As in the collapse problem, no condition is imposed at $x_{\min}$.

6 The Newton solver — `sc_newton_cs.m`

After collocation, the differential equations and boundary conditions form a nonlinear algebraic system

$$\mathbf{R}(\mathbf{u}) = \mathbf{0}, \quad (28)$$

where $\mathbf{u}$ is defined in Eq. (20). The file `sc_newton_cs.m` solves this system using a damped Newton iteration [4].

6.1 Complex-step Jacobian

The Jacobian

$$J = \frac{\partial \mathbf{R}}{\partial \mathbf{u}} \quad (29)$$

is evaluated with the complex-step method [5]. For a residual function that is analytic in its arguments,

$$\frac{\partial R_i}{\partial u_j} \approx \frac{\text{Im}[R_i(\mathbf{u} + ih \mathbf{e}_j)]}{h}, \quad (30)$$

where $\mathbf{e}_j$ is the j -th unit vector and h is a small real step ($h = 10^{-30}$ in the code). The implementation uses

```
1 for j = 1:n
2     up = u;
3     up(j) = up(j) + 1i * h;
4     J(:,j) = imag(resfun(up)) / h;
5 end
```

The complex-step method involves no subtraction of nearly equal numbers, so it avoids the cancellation error that limits ordinary finite differences and yields Jacobians accurate to near machine precision [5]. The Jacobian has dimension

$$2(N+1) \times 2(N+1). \quad (31)$$

6.2 Row scaling

The residual vector contains equations with different numerical scales. The implementation scales each residual row by the largest-magnitude entry in the corresponding Jacobian row:

```
1 rs = max(abs(J), [], 2);
2 rs(rs < 1e-30) = 1;
3
4 Js = J ./ rs;
5 Rs = R ./ rs;
```

If r_i denotes the scaling factor for row i , the scaled Newton system is

$$\text{diag}(1/r_i) J \delta \mathbf{u} = -\text{diag}(1/r_i) \mathbf{R}. \quad (32)$$

The same non-singular row scaling is applied to both sides, so the solution of this linear system is unchanged. The scaling does, however, improve the balance between equations of different magnitudes. (Because of the Tikhonov term below, the step actually computed is not exactly invariant under the scaling; with $\epsilon = 10^{-10}$ the effect is small.)

6.3 Tikhonov-regularized least-squares Newton step

The Newton correction is obtained from the regularized least-squares problem [6]

$$\min_{\delta \mathbf{u}} \|J_s \delta \mathbf{u} + \mathbf{R}_s\|_2^2 + \epsilon \|\delta \mathbf{u}\|_2^2, \quad (33)$$

which is equivalent to the augmented system

$$\begin{bmatrix} J_s \\ \sqrt{\epsilon} I_n \end{bmatrix} \delta \mathbf{u} = - \begin{bmatrix} \mathbf{R}_s \\ \mathbf{0} \end{bmatrix}, \quad (34)$$

where I_n is the $n \times n$ identity matrix with $n = 2(N + 1)$. The code solves this system directly (ϵ is `eps_reg`, not Octave's `eps`):

```
1 du = -[Js; sqrt(eps_reg)*eye(n)] ...
2       \ [Rs; zeros(n,1)];
```

This avoids forming the normal equations $J_s^T J_s + \epsilon I_n$, which would generally worsen conditioning [7]. The regularization parameter is $\epsilon = 10^{-10}$. It does not change the target equations; it only stabilizes the linear solve when the Jacobian is poorly conditioned, as can happen near the critical curve. When the Jacobian is well conditioned, the step approaches the ordinary Newton correction.

6.4 Backtracking line search

The Newton correction is damped by a backtracking line search [8]. Starting from $\lambda = 1$, the code halves the step (at most 30 times) until the residual satisfies

```
1 lam = 1;
2 for k = 1:30
3     if norm(resfun(u + lam*du)) ...
4         < nR*(1 - 1e-4*lam)
5         break;
6     end
7     lam = lam/2;
8 end
9
10 u = u + lam*du;
```


The acceptance condition is

$$\|\mathbf{R}(\mathbf{u} + \lambda \delta \mathbf{u})\| < \|\mathbf{R}(\mathbf{u})\| (1 - c\lambda), \quad c = 10^{-4}. \quad (35)$$

The line search prevents a large Newton step from leaving the basin of convergence [8].

6.5 Convergence criterion

The nonlinear iteration is stopped when

$$\frac{\|\mathbf{R}(\mathbf{u}^k)\|}{\|\mathbf{R}(\mathbf{u}^0)\|} < \text{tol}, \quad \text{tol} = 10^{-10}. \quad (36)$$

This criterion measures only the algebraic convergence of the Newton iteration, namely the reduction of the residual relative to its initial value. It says nothing about the discretization error of the collocation scheme or about the error caused by truncating the domain at $x_{\min}$ and $x_{\max}$. A small residual therefore does not establish agreement with Shu's tabulated values; that comparison is made separately in Sec. 11.

7 Grid refinement and continuation

The solver refines the grid in the polynomial degree N and uses continuation in the parameter A . Continuation in a parameter and in the grid resolution is a standard technique for extending the convergence basin of a Newton iteration [4, 9].

7.1 Grid refinement in N

The polynomial degree is increased through

```
1 Ns = [30, 60, 120];
```

At each stage, the solution from the previous grid is interpolated to the new grid in the coordinate τ and used as the initial guess:

$$\mathbf{u}_0^{(k+1)} = \mathcal{P}_{N_k \rightarrow N_{k+1}} \left[\mathbf{u}^{(k)} \right]. \quad (37)$$

This avoids restarting the nonlinear solve from a crude initial guess at every polynomial degree. Because the loop over N is embedded inside `sc_solve_collapse`, the sequence $N = 30 \rightarrow 60 \rightarrow 120$ is repeated for every value of A . Each Newton solve typically requires only a small number of iterations at each stage, since the initial guess is already close to the converged solution [2].

7.2 Continuation in A

For the collapse solutions, the solver proceeds through

```
1 A_list = [2.2 2.4 2.6 2.8 3.0 3.2 3.4 3.6 3.8 4.0];
```

The solution for one value of A initializes the next. Here the code interpolates in x with cubic splines:

```
1 v0 = interp1(x_prev, v_prev, x, 'spline');
2 a0 = interp1(x_prev, alpha_prev, x, 'spline');
3 u = [v0(:); log(a0(:))];
```

($a0$ is the interpolated density α , not the sound speed a .) This keeps successive nonlinear problems close to one another and makes the solutions for larger A easier to obtain [9].

7.3 Initial guess

For the first collapse calculation and for the expansion-wave calculation, the solver constructs an initial approximation from analytic limiting forms. The free-fall approximation is

$$\alpha_{\text{ff}}(x) = \sqrt{\frac{m_0}{2x^3}}, \quad v_{\text{ff}}(x) = -\sqrt{\frac{2m_0}{x}}, \quad (38)$$

and the large- x asymptotic approximation is

$$\alpha_{\infty}(x) = \frac{A}{x^2}, \quad v_{\infty}(x) = -\frac{A-2}{x}. \quad (39)$$

For the collapse calculation, the two forms are blended with the weight

$$\beta(x) = \frac{x^2}{1+x^2}, \quad (40)$$

where the density is blended in log space:

$$\ln \alpha_0 = (1-\beta) \ln \alpha_{\text{ff}} + \beta \ln \alpha_{\infty}, \quad (41)$$

$$v_0 = (1-\beta)v_{\text{ff}} + \beta v_{\infty}. \quad (42)$$

For the expansion-wave calculation, the outer form is the hydrostatic solution

$$\alpha_{\text{st}} = \frac{2}{x^2}, \quad (43)$$

and the densities are blended linearly in α with the weight $w_b = x/x_{\text{max}}$ (with $x_{\text{max}} = 1$), so that

$$\alpha_0 = (1-w_b)\alpha_{\text{ff}} + w_b\alpha_{\text{st}}, \quad v_0 = (1-w_b)v_{\text{ff}}; \quad (44)$$

the logarithm is then taken to form the initial w .

8 Method and code provenance

The implementation combines published scientific content with independent numerical implementation. The similarity equations, asymptotic conditions, critical-point conditions, and reference tabulated values are attributed to Shu (1977) [1]. The Chebyshev differentiation matrix follows the algorithm and code structure of Trefethen’s Program 6 [2]. The nonlinear solver uses standard Newton iteration [4], complex-step differentiation [5], Tikhonov regularization [6], linear-algebra operations following standard numerical practice [7], Armijo backtracking [8], and continuation methods [9].

The present source should therefore be understood as an *independent numerical implementation of the published equations using a different numerical formulation*, rather than as a transcription or recovery of Shu’s original software. In particular, Shu’s published calculation uses four-step Runge–Kutta integration, whereas the present implementation uses Chebyshev–Gauss–Lobatto collocation and a nonlinear Newton solve. The comparison is consequently a cross-code numerical validation rather than a software-reproduction claim.

AI-assisted tools were used during development of the implementation and this documentation. The mathematical formulation, numerical strategy, boundary conditions, computational experiments, and validation criteria were reviewed and tested by the researcher responsible for the reproduction. The use of AI assistance is therefore treated as part of the provenance of the development process; numerical validity is assessed by the documented equations, executable implementation, convergence checks, and quantitative comparison with Shu (1977).

For clarity, the provenance chain used throughout this document is

$$\boxed{\begin{aligned} &\text{published equations and reference values} \rightarrow \text{numerical reformulation} \rightarrow \text{spectral residuals} \\ &\rightarrow \text{Jacobian/Newton solve} \rightarrow \text{physical quantities} \rightarrow \text{quantitative validation.} \end{aligned}} \quad (45)$$

This distinction between scientific source, numerical-method reference, implementation, and validation result is intentional: it makes clear which parts of the reproduction originate in the published paper and which parts are methodological choices of the present implementation.

9 Equation-to-code traceability

Table [A1](#) maps each mathematical object to the code that implements it, with equation numbers.

Mathematical object	Equation/source	Implementation	Reference
Similarity ansatz and variable $x = r/(at)$	(1)	Used throughout the solver; numerical coordinate is $\tau = \ln x$.	Shu (1977) [1]
Similarity ODE for v	(2)	Residual R1 in <code>sc_res_collapse.m</code> and <code>sc_res_expwave.m</code> .	Shu (1977) [1]
Similarity ODE for α	(3)	Reformulated as the $w = \ln \alpha$ residual in the two residual functions.	Shu (1977) [1]
Mass relation $m = x^2 \alpha(x - v)$	(4)	Used by <code>shu1977_spectral_octave.m</code> to compute $m(x)$ and m_0 .	Shu (1977) [1]
Logarithmic coordinate $\tau = \ln x$	(7)	Constructed in the solver routines; the differentiation matrix acts on τ .	Boyd (2001) [3]
Log-density $w = \ln \alpha$	(10)	The state vector stores w ; $\alpha = e^w$ is reconstructed in the residuals.	Present numerical formulation
Chebyshev–Gauss–Lobatto nodes	(14)	<code>sc_cheb.m</code> constructs the nodes and differentiation matrix.	Trefethen (2000), Program 6 [2]
Derivative $d/d\xi$	(15)	Matrix D returned by <code>sc_cheb.m</code> .	Trefethen (2000) [2]
Derivative $d/d\tau$	(17), (18)	$Dt = D * 2 / (t_{\max} - t_{\min})$ in the residual functions.	Spectral mapping; cf. Trefethen (2000) [2]
Collapse outer boundary conditions	(25), (26)	Boundary residuals in <code>sc_res_collapse.m</code> .	Shu (1977) [1]
Critical-point conditions	(27)	Boundary residuals in <code>sc_res_expwave.m</code> .	Shu (1977) [1]
Jacobian $J = \partial \mathbf{R} / \partial \mathbf{u}$	(30)	Complex-step differentiation in <code>sc_newton_cs.m</code> .	Squire & Trapp (1998) [5]
Scaled Newton system	(32)	Row scaling in <code>sc_newton_cs.m</code> .	Dennis & Schnabel (1996); Golub & Van Loan (2013) [4, 7]
Regularized Newton correction	(33), (34)	Tikhonov-regularized least-squares solve in <code>sc_newton_cs.m</code> .	Tikhonov (1963) [6]
Backtracking line search	(35)	Damped Newton update in <code>sc_newton_cs.m</code> .	Armijo (1966) [8]
Grid refinement in N	(37)	<code>sc_solve_collapse.m</code> and <code>sc_solve_expwave.m</code> .	Present refinement procedure
Continuation in A	Sec. 7.2	<code>sc_solve_collapse.m</code> .	Allgower & Georg (1990) [9]
Comparison with Shu’s Table 1	Published Table 1	<code>shu1977_spectral_octave.m</code> .	Shu (1977) [1]
Comparison with Shu’s Table 2	Published Table 2	<code>shu1977_spectral_octave.m</code> .	Shu (1977) [1]
Validation and error analysis	Sec. 11	Main driver; recorded in the numerical validation output.	Present cross-code validation

Table A1: Equation-to-code traceability, including the source of each mathematical ingredient or numerical algorithm.

This traceability lets a reader follow each mathematical expression through to the numerical operation that evaluates it.

10 The main driver — `shu1977_spectral_octave.m`

The main driver `shu1977_spectral_octave.m` connects the solver components and compares the results with Shu (1977) [1]. It performs three tasks:

1. **Comparison with Shu’s Table 1 (collapse solutions).** For

$$A = 2.2, 2.4, \dots, 4.0,$$

the driver calls `sc_solve_collapse`, obtains the corresponding solution, and evaluates

$$m_0 \approx m(x_{\min}) = x_{\min}^2 \alpha(x_{\min}) [x_{\min} - v(x_{\min})]. \quad (46)$$

This is an approximation, since $m(x_{\min}) \neq m_0$ for finite $x_{\min}$; the truncation error is estimated by varying $x_{\min}$ (see Sec. 11). The computed values are compared with those printed in Shu’s Table 1.

2. **Comparison with Shu’s Table 2 (expansion-wave solution).** The expansion-wave calculation uses the critical-point boundary conditions (27). The solution is evaluated over

$$0.001 \leq x \leq 1$$

and compared with the values reported in Shu’s Table 2, with the numerical solution interpolated onto the x -values of the published table where necessary.

3. **Physical reconstruction.** The driver reconstructs the physical density and velocity fields from the similarity solution. For the expansion-wave solution, the self-similar solution is used inside the expansion wave, and the singular isothermal sphere is used outside it:

$$\rho = \frac{a^2}{2\pi Gr^2}, \quad u = 0. \quad (47)$$

The final version of the driver writes the numerical results (Tables 1 and 2, the error analysis, and the validation summary) to the text file `shu1977_results_spectral_full.txt`; it does not produce figures. Figure production is handled separately by the physical reconstruction utility.

11 Numerical validation

The test is whether the spectral solution reproduces the values published by Shu (1977) [1]. For m_0 in Shu’s Table 1, the maximum relative difference is 0.293% (at $A = 2.20$). For Shu’s Table 2, the maximum relative differences are

$$0.380\% \text{ for } \alpha, \quad 0.618\% \text{ for } -v, \quad 0.442\% \text{ for } m$$

at $x = 0.20$, $x = 0.95$, and $x = 0.25$ respectively. The critical-point values $v(x=1) = 0$ and $\alpha(x=1) = 2$ are imposed as boundary conditions (27), and $m(x=1) = 2$ follows from Eq. (4); they are therefore not part of the validation. The observed differences can be understood as follows.

- The maximum relative difference in Table 1, 0.293% at $A = 2.20$, and the maxima for α and m in Table 2, 0.380% and 0.442%, are all within the rounding interval of the paper’s three-significant-figure entries (at most 0.5%).

- The maximum relative difference for $-v$, 0.618%, occurs at $x = 0.95$, where the paper reports $-v = 0.051$ with only two significant figures. The rounding interval for this printed value is ± 0.0005 , i.e. a relative interval of up to 0.98%. The spectral value is 0.05130, which lies inside the paper’s rounding interval. The apparent 0.618% difference is therefore entirely due to the paper’s printed precision at this point.
- The residual of the Newton iteration converges to 10^{-10} , so the algebraic residual of the spectral solution is not the source of the observed differences. The remaining discrepancies are dominated by (i) the rounding of the paper’s tabulated values and (ii) the paper’s own numerical accuracy, which is limited by the four-step Runge–Kutta integration that Shu uses with a step size that may not fully resolve the critical region near $x = 1$. Both effects are of the same order as the observed differences, so the spectral solution may be considered converged relative to the precision at which the paper reports its results.

12 Summary of the method

The numerical procedure can be summarized as follows:

1. Rewrite Shu’s similarity equations in the variables $\tau = \ln x$ and $w = \ln \alpha$, and map the τ interval onto $[-1, 1]$.
2. Discretize v and w by their nodal values on the Chebyshev–Gauss–Lobatto grid, using the differentiation matrix to form the nonlinear residual equations, with the outer-node residuals replaced by the boundary conditions.
3. Solve the system with a scaled, Tikhonov-regularized, damped Newton iteration whose Jacobian is computed by complex-step differentiation.
4. Refine the grid in N and continue in A .
5. Recover α , m and the physical variables, and compare with Shu’s Tables 1 and 2.

Table A2 contrasts this procedure with that of Shu (1977).

	Shu (1977)	This work
Formulation	ODEs in x, v, α	ODEs in $\tau = \ln x, v, w = \ln \alpha$
Initialization	Asymptotic expansions	Analytic blend of limiting forms
Discretization	Four-step Runge–Kutta integration	Chebyshev collocation [2]
Nonlinear solver	— (integration of an initial-value problem)	Complex-step-Jacobian Newton iteration [4, 5]

Table A2: Methodological differences between Shu (1977) and the present calculation.

13 Source-file summary

File	Mathematical / numerical role
<code>sc_cheb.m</code>	Constructs the Chebyshev–Gauss–Lobatto nodes and differentiation matrix D ; the implementation follows Trefethen [2, Program 6].
<code>sc_res_collapse.m</code>	Constructs the log-transformed collapse residual equations and imposes the outer asymptotic boundary conditions.
<code>sc_res_expwave.m</code>	Constructs the log-transformed expansion-wave residual equations and imposes the critical-point conditions $v(x=1) = 0$ and $\alpha(x=1) = 2$.
<code>sc_newton_cs.m</code>	Computes the complex-step Jacobian [5] and solves the nonlinear collocation system using scaled, Tikhonov-regularized Newton steps [6] with Armijo backtracking [8].
<code>sc_solve_collapse.m</code>	Controls the grid refinement in N and the continuation in A for the collapse family.
<code>sc_solve_expwave.m</code>	Solves the limiting expansion-wave problem using the critical-point boundary conditions.
<code>shu1977_spectral_octave.m</code>	Main driver: comparison with Shu’s Tables 1 and 2, validation and error analysis, and output generation.

Table A3: Summary of the source files and their mathematical roles.

14 Reproducibility statement

The calculation can be repeated with the following information.

- Software: GNU Octave 10.3.0 (or compatible).
- Code: the main driver `shu1977_spectral_octave.m` together with the helper files `sc_cheb.m`, `sc_res_collapse.m`, `sc_res_expwave.m`, `sc_newton_cs.m`, `sc_solve_collapse.m`, and `sc_solve_expwave.m`.
- Domain: $x_{\min} = 10^{-3}$; $x_{\max} = 20$ (collapse) and $x_{\max} = 1$ (expansion wave).
- Polynomial degrees: $N = 30, 60, 120$.
- Collapse parameters: $A = 2.2, 2.4, \dots, 4.0$.
- Solver parameters: $h = 10^{-30}$ (complex step), $\epsilon = 10^{-10}$ (Tikhonov), $\text{tol} = 10^{-10}$, $c = 10^{-4}$, at most 30 step halvings per iteration.
- Run time: the full calculation (10 collapse cases plus one expansion-wave case) completes in a few seconds on a standard desktop machine.

References

- [1] Shu, F. H. 1977, *The Astrophysical Journal*, **214**, 488–497. “Self-similar collapse of isothermal spheres and star formation.”
- [2] Trefethen, L. N. 2000, *Spectral Methods in MATLAB*, Software, Environments, and Tools, vol. 10 (Philadelphia: SIAM). The Chebyshev differentiation matrix `sc_cheb.m` used in this work is based on Program 6 of this book.

- [3] Boyd, J. P. 2001, *Chebyshev and Fourier Spectral Methods*, 2nd ed. (Mineola, NY: Dover Publications).
- [4] Dennis, J. E., Jr., & Schnabel, R. B. 1996, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*, Classics in Applied Mathematics, vol. 16 (Philadelphia: SIAM).
- [5] Squire, W., & Trapp, G. 1998, “Using complex variables to estimate derivatives of real functions,” *SIAM Review*, **40**(1), 110–112.
- [6] Tikhonov, A. N. 1963, “Solution of incorrectly formulated problems and the regularization method,” *Soviet Mathematics Doklady*, **4**, 1035–1038.
- [7] Golub, G. H., & Van Loan, C. F. 2013, *Matrix Computations*, 4th ed. (Baltimore: Johns Hopkins University Press).
- [8] Armijo, L. 1966, “Minimization of functions having Lipschitz continuous first partial derivatives,” *Pacific Journal of Mathematics*, **16**(1), 1–3.
- [9] Allgower, E. L., & Georg, K. 1990, *Numerical Continuation Methods: An Introduction*, Springer Series in Computational Mathematics, vol. 13 (Berlin: Springer-Verlag).